

05 逆矩阵

5.3 手解逆矩阵

用伴随矩阵法，一步步手算 3×3 方阵的逆矩阵



鸢尾花书·《线性代数不难》 | 开源代码与书稿: <https://github.com/Visualize-ML>

本节你将掌握的核心技能

1 方阵可逆的前提是行列式不为零

3 每个余子矩阵求行列式，结果为余子式

5 代数余子式构成代数余子式矩阵

7 逆矩阵可通过伴随矩阵除以行列式得到

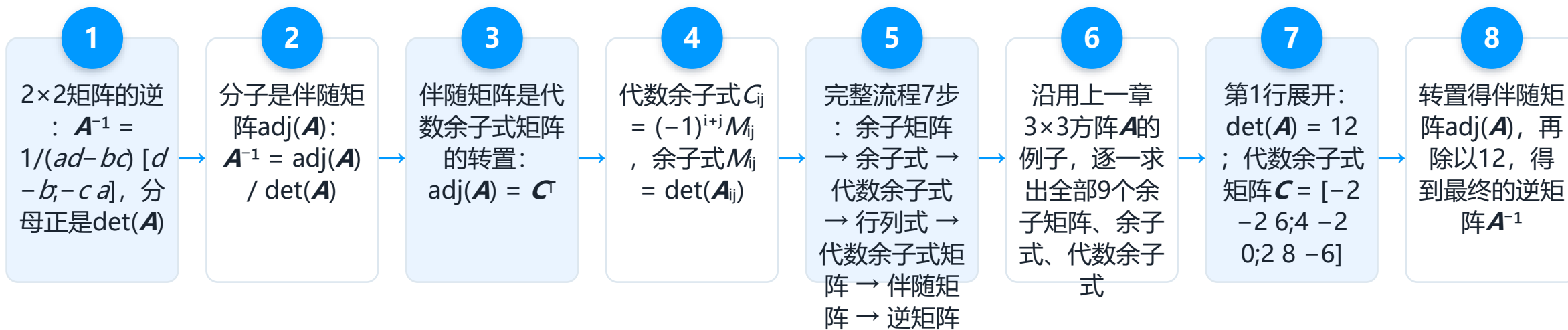
2 删除对应行列，构造每个元素的余子矩阵

4 根据位置符号调整余子式，得到代数余子式

6 代数余子式矩阵进行转置，得到伴随矩阵

8 复习：拉普拉斯展开计算行列式

本节知识脉络



从2×2矩阵的逆矩阵公式说起

$$\mathbf{A} = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \Rightarrow \mathbf{A}^{-1} = 1/(ad-bc) \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}$$

- 本章前文提过，方阵 $\mathbf{A}$ 可逆的充分必要条件是 $\mathbf{A}$ 的行列式不为0；上式中分母 $ad-bc$ 正是 $\mathbf{A}$ 的行列式 $\det(\mathbf{A})$
- 分母是行列式，那么分子是什么？这个分子叫 $\mathbf{A}$ 的伴随矩阵（adjoint of a matrix），记作 $\text{adj}(\mathbf{A})$

逆矩阵 = 伴随矩阵 ÷ 行列式

$$\mathbf{A}^{-1} = \text{adj}(\mathbf{A}) / \det(\mathbf{A})$$

$$\mathbf{A}^{-1} = \text{adj}(\mathbf{A}) / \det(\mathbf{A})$$

- 如果矩阵 $\mathbf{A}$ 可逆，它的逆矩阵可以写成伴随矩阵除以行列式——为了计算逆矩阵，我们要先计算行列式、再计算伴随矩阵
- 伴随矩阵的定义为 $\text{adj}(\mathbf{A}) = \mathbf{C}^\top$ ，其中 $\mathbf{C}$ 叫做代数余子式矩阵；显然，伴随矩阵是代数余子式矩阵的转置

代数余子式矩阵由每个元素的代数余子式构成

$$C_{ij} = (-1)^{i+j} M_{ij}, \quad M_{ij} = \det(A_{ij})$$

- 每个代数余子式，是删除该元素所在的行、列后，先求余子矩阵 A_{ij} 的行列式（余子式 M_{ij} ），再根据位置引入符号调整得到的
- 余子矩阵 A_{ij} 是矩阵 A 中去掉第 i 行、第 j 列后剩余元素构成的矩阵；余子式 M_{ij} 就是这个余子矩阵的行列式

反过来看，整个求解逆矩阵的过程

a) 计算行列式展开时用到所有行或列，提取余子矩阵（方阵）

b) 根据余子矩阵（方阵）计算原矩阵所有元素的余子式（标量）

c) 计算每个元素的代数余子式（标量），即调整符号后的余子式

d) 根据拉普拉斯展开，计算行列式

e) 根据 c) 的结果，构造代数余子式矩阵

f) 代数余子式矩阵转置，得到伴随矩阵

g) 最后，用行列式倒数乘上伴随矩阵，得到逆矩阵

注意 计算行列式只需在特定行、列展开提取余子矩阵；构造代数余子式矩阵则需要提取所有元素的余子矩阵

给定 3×3 方阵A，一步步计算逆矩阵

$$A = [1 \ 2 \ 3; 3 \ 0 \ 1; 1 \ 2 \ 1]$$

- 这个例子沿用上一节手解行列式的 3×3 方阵——大部分运算都来自那一节计算行列式的过程，正好帮助我们复习余子式、代数余子式、拉普拉斯展开
- 接下来，我们提取全部9个元素的余子矩阵，逐一计算余子式与代数余子式

9个元素的余子矩阵、余子式与代数余子式

元素	余子矩阵 $A_{i,j}$	余子式 $M_{i,j}$	代数余子式 $C_{i,j}$
$a_{1,1}$	$\begin{bmatrix} 0 & 1; 2 & 1 \end{bmatrix}$	-2	-2
$a_{1,2}$	$\begin{bmatrix} 3 & 1; 1 & 1 \end{bmatrix}$	2	-2
$a_{1,3}$	$\begin{bmatrix} 3 & 0; 1 & 2 \end{bmatrix}$	6	6
$a_{2,1}$	$\begin{bmatrix} 2 & 3; 2 & 1 \end{bmatrix}$	-4	4
$a_{2,2}$	$\begin{bmatrix} 1 & 3; 1 & 1 \end{bmatrix}$	-2	-2
$a_{2,3}$	$\begin{bmatrix} 1 & 2; 1 & 2 \end{bmatrix}$	0	0
$a_{3,1}$	$\begin{bmatrix} 2 & 3; 0 & 1 \end{bmatrix}$	2	2
$a_{3,2}$	$\begin{bmatrix} 1 & 3; 3 & 1 \end{bmatrix}$	-8	8
$a_{3,3}$	$\begin{bmatrix} 1 & 2; 3 & 0 \end{bmatrix}$	-6	-6

仅用第1行展开计算行列式

$$\det(\mathbf{A}) = a_{1,1}C_{1,1} + a_{1,2}C_{1,2} + a_{1,3}C_{1,3} = 1 \times (-2) + 2 \times (-2) + 3 \times 6 = 12$$

- 代入上一页表格中第1行对应的代数余子式 $C_{1,1} = -2$ 、 $C_{1,2} = -2$ 、 $C_{1,3} = 6$ ，即可求出 $\det(\mathbf{A}) = 12$
- $\det(\mathbf{A}) \neq 0$ ，说明 $\mathbf{A}$ 可逆——接下来只需要构造代数余子式矩阵、求出伴随矩阵，就能得到逆矩阵

根据9个代数余子式，构造矩阵C

$$C = [C_{1,1} \ C_{1,2} \ C_{1,3}; C_{2,1} \ C_{2,2} \ C_{2,3}; C_{3,1} \ C_{3,2} \ C_{3,3}] = [-2 \ -2 \ 6; 4 \ -2 \ 0; 2 \ 8 \ -6]$$

- 把表格中9个代数余子式按原来的行、列位置排列，就得到了代数余子式矩阵 **C**
- 下一步，只需要把 **C** 转置，就能得到伴随矩阵 $\text{adj}(\mathbf{A})$

转置得到伴随矩阵

$$\text{adj}(\mathbf{A}) = \mathbf{C}^T$$

$$\text{adj}(\mathbf{A}) = \mathbf{C}^T = [-2 \ 4 \ 2; -2 \ -2 \ 8; 6 \ 0 \ -6]$$

- 根据伴随矩阵的定义 $\text{adj}(\mathbf{A}) = \mathbf{C}^T$ ，把代数余子式矩阵 $\mathbf{C}$ 的行、列互换，就得到伴随矩阵
- 请大家留意： $\mathbf{C}$ 的第一行 $[-2 \ -2 \ 6]$ 变成了 $\text{adj}(\mathbf{A})$ 的第一列 $[-2; -2; 6]$ ，这正是转置的效果

计算逆矩阵

$$\mathbf{A}^{-1} = \text{adj}(\mathbf{A}) / \det(\mathbf{A})$$

$$\mathbf{A}^{-1} = (1/12) \times \begin{bmatrix} -2 & 4 & 2 \\ -2 & -2 & 8 \\ 6 & 0 & -6 \end{bmatrix}$$

- 代入 $\det(\mathbf{A}) = 12$ 和 $\text{adj}(\mathbf{A})$ ，就得到了 $\mathbf{A}$ 的逆矩阵——用行列式的倒数乘上伴随矩阵
- 可以用 $\mathbf{A} @ \mathbf{A}^{-1} = \mathbf{I}$ 来验证结果是否正确，这也是判断手算有没有出错的好方法

LA_05_03_01.ipynb · 从公式到代码

- a) 实现余子矩阵的提取：用`numpy.delete()`依次删除指定的行、列
- b) 计算每个余子矩阵的行列式，得到9个余子式
- c) 根据位置符号 $(-1)^{i+j}$ 调整余子式，得到9个代数余子式
- d) 用拉普拉斯展开（第1行）求出行列式`det(A)`
- e) 把9个代数余子式排列成代数余子式矩阵C，再转置得到伴随矩阵`adj(A)`
- f) 用`adj(A)`除以`det(A)`，得到逆矩阵，并与`numpy.linalg.inv()`的结果比较

伴随矩阵法：公式清晰，但计算量大

$$A^{-1} = \text{adj}(A) / \det(A) = C^T / \det(A)$$

- 伴随矩阵法把求逆矩阵拆解成：提取余子矩阵、计算余子式、计算代数余子式、求行列式、转置得伴随矩阵——每一步都有明确的几何、代数含义
- 但当矩阵规模变大时，手算代数余子式矩阵的运算量会迅速增加；从计算效率、稳定性、数值精度等角度来看，建议优先使用numpy.linalg.inv()等现成工具
- 本册最后还会介绍另一种手解方法——初等行变换法，两种方法殊途同归

本节关键术语

伴随矩阵 *Adjoint of a Matrix, $\text{adj}(A)$* —— 代数余子式矩阵的转置, 满足 $A^{-1} = \text{adj}(A)/\det(A)$

余子矩阵 *Minor Matrix A_{ij}* —— 矩阵A去掉第i行、第j列后剩余元素构成的矩阵

代数余子式 *Cofactor C_{ij}* —— 加了位置符号的余子式, $C_{ij} = (-1)^{i+j}M_{ij}$

2×2矩阵的逆 $A^{-1} = 1/(ad-bc) [d \ -b; -c \ a]$ —— 伴随矩阵法在2×2场景下的简化形式

初等行变换法 *Row Reduction Method* —— 另一种手解逆矩阵的方法, 本册最后章节介绍

代数余子式矩阵 *Cofactor Matrix C* —— 由矩阵中每个元素对应的代数余子式构成

余子式 *Minor M_{ij}* —— 余子矩阵 A_{ij} 的行列式, $M_{ij} = \det(A_{ij})$

拉普拉斯展开 *Laplace Expansion* —— 沿任意一行或一列, 用代数余子式展开计算行列式

转置 *Transpose* —— 把矩阵的行、列互换, $\text{adj}(A) = C^T$ 正是转置的应用

numpy.linalg.inv() NumPy提供的现成求逆函数, 计算效率、稳定性、数值精度均优于手算

配套代码与延伸阅读

配套代码 • LA_05_03_01.ipynb

完整实现余子矩阵提取、余子式与代数余子式计算、行列式求解、伴随矩阵求取与逆矩阵计算

延伸阅读 • `numpy.linalg.inv()`

从计算效率、稳定性、数值精度等角度，建议优先使用NumPy提供的线性代数函数求逆

延伸阅读 • 初等行变换法

另一种手解逆矩阵的方法，本册最后章节将系统介绍

开源仓库

全书代码与图像素材：<https://github.com/Visualize-ML>

习题 Q1 – Q8

Q1 用本节介绍的方法计算如下 2×2 矩阵的逆: $A = [a,b;c,d]$ 。

Q3 请自学使用`time.time()`, 比较`numpy.linalg.inv()`、自定义函数求解矩阵逆的运算时间。

Q5 写一个函数`get_minor_matrix(A, i, j)`, 用`numpy.delete()`删除第 i 行、第 j 列, 返回余子矩阵 $A_{i,j}$ 。

Q7 写一个函数`adjoint(A)`, 调用`cofactor_matrix(A)`并转置, 返回伴随矩阵 $\text{adj}(A)$ 。

Q2 用本节介绍的方法手解如下 3×3 矩阵的逆, 并用NumPy验证结果: $A = [1,0,1;0,1,1;1,1,0]$ 。

Q4 请自学初等行变换手解逆矩阵。

Q6 写一个函数`cofactor_matrix(A)`, 对方阵 A 的每个元素调用`get_minor_matrix()`, 构造出完整的代数余子式矩阵 C 。

Q8 写一个函数`inverse_by_adjoint(A)`, 结合`adjoint(A)`与`numpy.linalg.det(A)`, 实现完整的伴随矩阵法求逆, 并与`numpy.linalg.inv(A)`比较结果。

习题 Q9 – Q16

Q9 用本节方法手解 4×4 单位矩阵 I 的逆，验证结果是否仍是 I 。

Q11 写一个函数`verify_inverse(A)`，计算 $A @ \text{inverse_by_adjoint}(A)$ ，验证结果是否等于单位矩阵（容差 $1e-9$ ）。

Q13 写一个函数`random_invertible_3x3()`，随机生成一个 3×3 矩阵，并检查（必要时重新生成）确保其行列式不为0，再用伴随矩阵法求逆。

Q15 写一个函数`cofactor_sign(i, j)`，返回 $(-1)^{i+j}$ ，并用它验证本节9个代数余子式的符号是否正确。

Q10 构造一个 3×3 对角矩阵 $D = \text{diag}(2, 3, 4)$ ，用伴随矩阵法手解其逆，并与直接取倒数的结果比较。

Q12 对本节例子中的 3×3 矩阵 A ，分别用第2行、第3行展开计算 $\det(A)$ ，验证结果是否都等于12。

Q14 用SymPy对含符号变量的 3×3 矩阵求伴随矩阵 $\text{adj}(A)$ ，并与numpy数值结果做对比。

Q16 构造一个行列式为0的 3×3 矩阵，尝试用 $\text{adjoint}(A)/\det(A)$ 计算其逆，观察会发生什么，并解释原因。

习题 Q17 – Q24

Q17 写一个函数`inverse_2x2(a, b, c, d)`，直接套用 2×2 矩阵的逆矩阵公式，返回逆矩阵。

Q19 写一个函数`matrix_of_cofactors_check(A)`，验证代数余子式矩阵 C 是否满足 $A @ C.T = \det(A) \times I$ 。

Q21 写一个函数`adjoint_of_transpose(A)`，验证 $\text{adj}(A.T)$ 是否等于 $\text{adj}(A).T$ 。

Q23 写一个函数`inverse_via_cramer(A, b)`，用伴随矩阵/克莱姆法则的思路求解线性方程组 $Ax=b$ （提示： $x = A^{-1}b$ ）。

Q18 用`time.time()`比较伴随矩阵法与`numpy.linalg.inv()`在 5×5 随机矩阵上的运算时间差异。

Q20 对本节例子中的矩阵 A ，手算并验证：把 A 的某一行替换为另一行后，行列式是否变为0（对应奇异矩阵）。

Q22 构造一个 2×2 矩阵，比较直接套用公式 $A^{-1} = \text{adj}(A)/\det(A)$ 与伴随矩阵法逐步计算的结果是否一致。

Q24 结合本节所学，写一份简短总结（可以是代码注释形式），比较伴随矩阵法与`numpy.linalg.inv()`各自的优缺点。

10 个可深入探讨的 Prompt (1/2)

- 1 请解释为什么 2×2 矩阵的逆矩阵公式中，分母 $ad - bc$ 正是矩阵的行列式？
- 2 请解释伴随矩阵 $\text{adj}(A)$ 和代数余子式矩阵 C 之间的关系，为什么 $\text{adj}(A) = C^T$ 而不是 $\text{adj}(A) = C$ ？
- 3 请解释余子式 M_{ij} 和代数余子式 C_{ij} 的区别，为什么代数余子式需要引入 $(-1)^{i+j}$ 这个符号？
- 4 请解释为什么计算行列式时只需要展开一行或一列，而构造代数余子式矩阵却需要计算所有9个元素？
- 5 请解释伴随矩阵法求逆的完整流程，并说明每一步分别对应本节哪个公式？

10 个可深入探讨的 Prompt (2/2)

6

请解释为什么可以用 $A @ A^{-1} = I$ 来验证手算逆矩阵的结果是否正确？

7

请解释当矩阵规模从 2×2 扩大到 3×3 、 4×4 时，伴随矩阵法的计算量为什么会迅速增加？

8

请解释为什么从计算效率、稳定性、数值精度的角度，实际编程中更推荐使用 `numpy.linalg.inv()` 而不是手写伴随矩阵法？

9

请解释代数余子式矩阵 C 与上一节学过的行列式 $\det(A)$ 之间有什么联系？

10

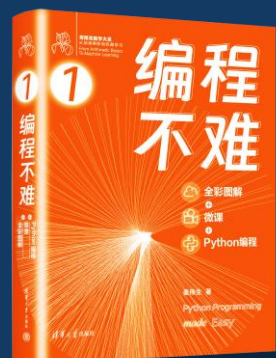
请结合本节所学，谈谈伴随矩阵法和初等行变换法这两种手解逆矩阵方法可能各自的优缺点？

数学

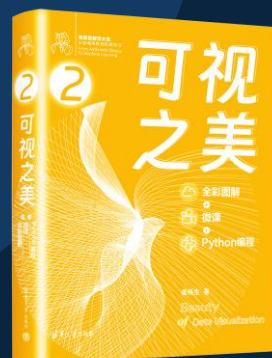
数学基础

线性代数

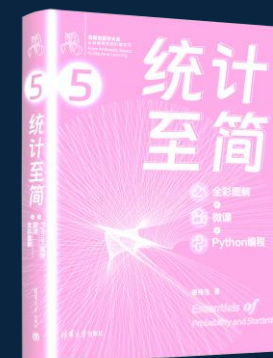
概率统计



Python编程



数据可视化



工具

数据分析
图和网络

回归、降维
分类、聚类

实践

